

Python Programming with Artificial Intelligence

Advanced Python Fundamentals

Chengwei QIN

Assistant Professor @ AI Thrust

Exceptions

- Exception Hierarchy, Exception Handling

Functions

- Fundamentals, Parameters and Arguments, Anonymous(Lambda) Functions



Exceptions

Error Message

The top part shows the context of the error — a stack traceback indicating where the error occurred

The last line shows the error type and a short description of what went wrong

```
Traceback (most recent call last):  
  File "/Users/sunying/Desktop/Python class/week1/class_27/  
project_1/mymodule.py", line 8, in <module>  
    print(cou) The statement The exact place  
NameError: name 'cou' is not defined
```

Two main categories of errors:

- **Syntax Errors:** Occur when the code is not valid Python (also called parsing errors).

```
>>> while True print('Hello world')
      File "<stdin>", line 1
        while True print('Hello world')
                        ^
SyntaxError: invalid syntax
```

- **Runtime Errors (Exceptions):** The code is syntactically correct, but errors occur while executing
 - Can be fatal, depending on how they are handled
 - Common causes include invalid input values, wrong usage of functions, or external issues such as network failures

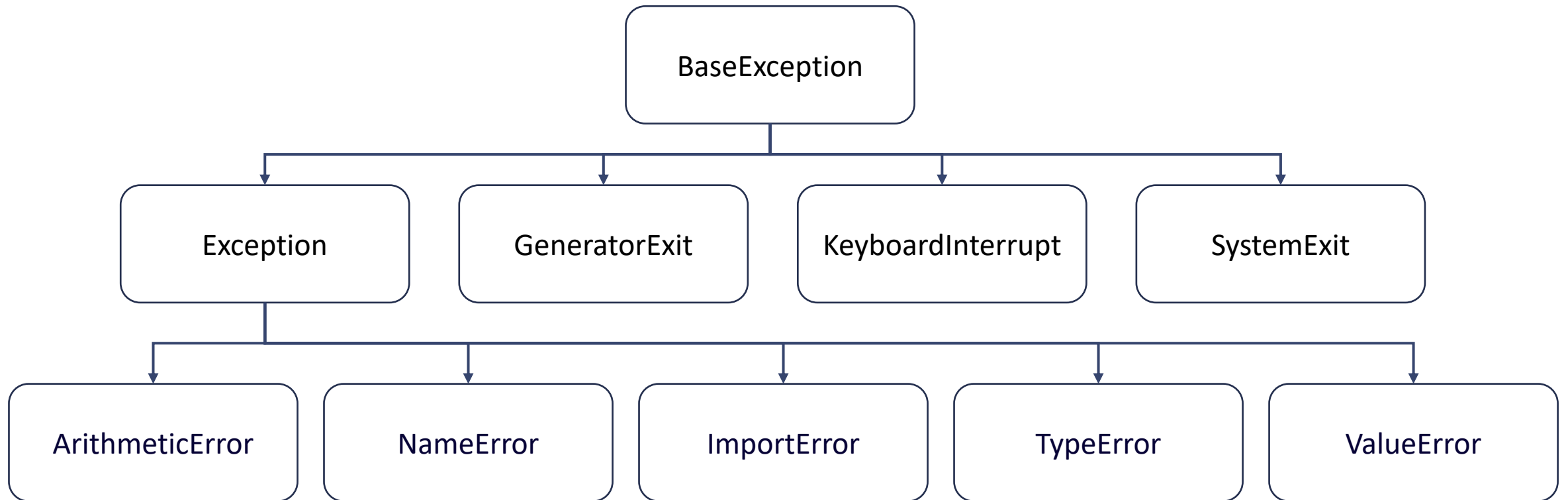
Python can raise many different types of errors during execution

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

These messages indicate the specific type of Exception that occurred

Exceptions

- In python, errors are represented as *exceptions*
- Since everything in Python is an object, exceptions are objects too
- Exceptions follow a hierarchical structure via *inheritance*



Exceptions

- ArithmeticError
 - FloatingPointError
 - OverflowError
 - ZeroDivisionError
 - AssertionError
 - AttributeError
 - BufferError
 - EOFError
 - ImportError
 - ModuleNotFoundError
 - LookupError
 - IndexError
 - KeyError
 - MemoryError
 - NameError
 - UnboundLocalError
- OSError
 - BlockingIOError
 - ChildProcessError
 - ConnectionError
 - BrokenPipeError
 - ConnectionAbortedError
 - ConnectionRefusedError
 - ConnectionResetError
 - FileExistsError
 - FileNotFoundError
 - InterruptedError
 - IsADirectoryError
 - NotADirectoryError
 - PermissionError
 - ProcessLookupError
 - TimeoutError
- ReferenceError
 - RuntimeError
 - NotImplementedError
 - RecursionError
 - StopIteration
 - StopAsyncIteration
 - SyntaxError
 - IndentationError
 - TabError
 - SystemError
 - TypeError
 - ValueError
 - UnicodeError
 - UnicodeDecodeError
 - UnicodeEncodeError
 - UnicodeTranslateError
 - Warning

Exception Handling

In Python, exceptions are raised when something unexpected occurs

- By default, an unhandled exception will terminate the program
- Python provides mechanisms to handle exceptions explicitly. So the program can recover or take alternative actions instead of crashing
- Keywords used in exception handling:
 - **try, except, else, finally, raise, assert**

```
try_stmt ::= try1_stmt | try2_stmt | try3_stmt
try1_stmt ::= "try" ":" suite
              ("except" [expression ["as" identifier]] ":" suite)+
              ["else" ":" suite]
              ["finally" ":" suite]
try2_stmt ::= "try" ":" suite
              ("except" "*" expression ["as" identifier] ":" suite)+
              ["else" ":" suite]
              ["finally" ":" suite]
try3_stmt ::= "try" ":" suite
              "finally" ":" suite
```

Try-Except Statement

Handle exceptions that may occur in specific parts of the code

- If an error occurs inside the **try** block, the corresponding **except** block is executed
- This prevents the program from crashing unexpectedly

Syntax

Exceptions that occur inside the try block are caught and handled by the corresponding except block

```
01. try:
02.     statement(s)
03. except:
04.     error-handling-statement(s)
```

The except block runs only when an exception is caught; if no exception occurs, it is skipped

Try-Except Statement

Example: A simple division function

```
def divide(n, m):  
    print('Let\'s start the Division!')  
    return n / m
```

✓ 3.9s

Python

Divide 4 by 1

```
print(divide(4, 1))
```

✓ 0.0s

```
Let's start the Division!  
4.0
```

Divide 4 by 0, error!

```
print(divide(4, 0))
```

⊗ 0.0s

```
Let's start the Division!
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
/var/folders/1p/9bcd1_s73b6cmjvy82yc_vr0000gn/T/ipykernel_99575/1158075214.py in <module>  
----> 1 print(divide(4, 0))  
  
/var/folders/1p/9bcd1_s73b6cmjvy82yc_vr0000gn/T/ipykernel_99575/3250101075.py in divide(n, m)  
      1 def divide(n, m):  
      2     print('Let\'s start the Division!')  
----> 3     return n / m  
  
ZeroDivisionError: division by zero
```

Try-Except Statement

Example

- Now add exception handling with try-except statement
- return a large number if divided by zero

```
def safe_divide(n, m):  
    try:  
        print('Let\'s start the Division!')  
        print('result:', n / m)  
    except:  
        print('result:', 1e100)
```

Run normally

```
safe_divide(4, 1)  
✓ 0.0s  
Let's start the Division!  
result: 4.0
```

The error handling statements got executed

```
safe_divide(4, 0)  
✓ 0.0s  
Let's start the Division!  
result: 1e+100
```

```
safe_divide(4, '2')  
✓ 0.0s  
Let's start the Division!  
result: 1e+100
```

What if other error happens?

Catch Specific Exceptions

Different errors can happen under different circumstances

- Example: output 1e100 when divide a number by a string

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except:  
        return 1E100  
  
print(safe_divide(1, '2'))
```

✓ 0.0s

1e+100

■ Catch Specific Exceptions

Handle only exception of a specific type

- Dividing an integer and a string raises a `TypeError`
- Divide by zero raises a `ZeroDivisionError`

Syntax

```
01. try:
02.     expression
03. except ErrorType:
04.     expression
```

Catch Specific Exceptions

Example: Handle only the ZeroDivisionError

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ZeroDivisionError:  
05.         return 1E100
```

```
print(safe_divide(2, 0))
```

✓ 0.1s

Python

1e+100

```
print(safe_divide(1, 2))
```

✓ 0.1s

Python

0.5

TypeError is not handled

```
print(safe_divide(1, '2'))
```

⊗ 0.2s

Python

TypeError

Traceback

ck (most recent call last)

Cell In[11], line 1

----> 1 print(safe_divide(1, '2'))

Cell In[8], line 3, in safe_divide(a, b)

1 def safe_divide(a, b):

2 try:

----> 3 return a / b

4 except ZeroDivisionError:

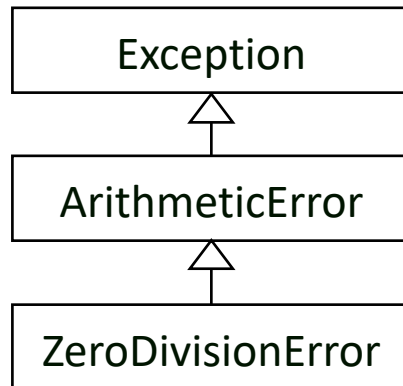
5 return 1E100

TypeError: unsupported operand type(s) for /: 'int' and 'str'

Catch Specific Exceptions

Inheritance relationships are important

- Example: A ZeroDivisionError is also an ArithmeticError and an Exception



```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ArithmeticError:  
05.         return 1E100
```

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ZeroDivisionError:  
05.         return 1E100
```

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except Exception:  
05.         return 1E100
```

Get the Exception

Access the raised exception object using the *as* keyword

Syntax

```
01. try:
02.     expression
03. except ErrorType as e:
04.     expression
```

A variable can be used to store the exception object.
It can be any valid identifier

Use this object to inspect or handle the error

Access the Exception

Access the raised exception object using the *as* keyword

Example: catch all exceptions and inspect their types

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except Exception as e:  
05.         return e
```

Large category

An exception object

- Get the type: `type()`
- Get the descriptions: `str()`

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except Exception as e:  
        return e  
  
a = safe_divide(1, 0)  
b = safe_divide(1, '2')
```

```
print(a, type(a))  
print(b, type(b))  
a
```

✓ 0.0s

```
division by zero <class 'ZeroDivisionError'>  
unsupported operand type(s) for /: 'int' and 'str' <class 'TypeError'>  
  
ZeroDivisionError('division by zero')
```

Handle Specific Exceptions

Handle different exceptions with different operations

- **Multiple except blocks** are allowed

Syntax

```
01. try:
02.     expression
03. except ErrorType as e:
04.     expression
05. except ErrorType as e:
06.     expression
```

Execute when: catch the desired type

Execute when:

- **previous desired type is not caught**
- Current desired type is caught
- Notice: **inheritance matters**

Handle Specific Exceptions

Example

Handle `TypeError` and `ZeroDivisionError` with different actions

```
01. def safe_divide(a, b):  
02.     try:  
03.         return a / b  
04.     except ZeroDivisionError:  
05.         return 1E100  
06.     except TypeError:  
07.         print('Not valid input')  
08.         return 0
```

```
print(safe_divide(2, 0))
```

✓ 0.1s

1e+100

```
print(safe_divide(1, '2'))
```

✓ 0.1s

Not valid input

0

Handle Exception

else statement

- Executes only if no exceptions are raised in the try block

Syntax

```
01. try:
02.     expression
03. except ErrorType [as e]:
04.     expression
05. except ErrorType [as e]:
06.     expression
07. else:
08.     expression
```

Execute when:
No exception happens

Handle Exception

else statement

- Example

Example

```
01. def safe_divide(a, b):  
02.     try:  
03.         c = a / b  
04.     except ZeroDivisionError:  
05.         return 1E100  
06.     except TypeError:  
07.         print('Not valid input')  
08.         return 0  
09.     else:  
10.         print('No exception')  
11.         return c
```

```
print(safe_divide(1, 2))
```

✓ 0.2s

Python

No exception

0.5

```
print(safe_divide(2, 0))
```

✓ 0.1s

Python

1e+100

```
print(safe_divide(1, '2'))
```

✓ 0.1s

Python

Not valid input

0

Handle Exception

finally statement

- The finally block is **always executed** after the try-except structure

Syntax

```
01. try:
02.     expression
03. except ErrorType [as e]:
04.     expression
05. except ErrorType [as e]:
06.     expression
07. else:
08.     expression
09. finally:
10.     expression
```

Execute anyway

- No matter catch exception or not

Handle Exception

finally statement

- The finally block is **always executed** after the try-except structure

Syntax

```
01. def safe_divide(a, b):  
02.     try:  
03.         c = a / b  
04.     except ZeroDivisionError:  
05.         c = 1E100  
06.     except TypeError:  
07.         print('Not valid input')  
08.         c = 0  
09.     else:  
10.         print('No exception')  
11.     finally:  
12.         print('Ended')  
13.         return c
```

```
print(safe_divide(1, 2))  
✓ 0.2s Python  
No exception  
Ended  
0.5
```

```
print(safe_divide(2, 0))  
✓ 0.1s Python  
Ended  
1e+100
```

```
print(safe_divide(1, '2'))  
✓ 0.1s Python  
Not valid input  
Ended  
0
```

Summarize

```
try:
    print("try something here")
except:
    print("this happens only if it fails")
else:
    print("this happens only if it succeeds")
finally:
    print("this happens no matter what")
```

✓ 0.2s

try something here
this happens only if it succeeds
this happens no matter what

Try: catch exception

Except: Handle exception

Else: Execute when no exception

Finally: Execute anyway

Raising Exceptions

Use the raise statement to explicitly trigger a specified exception

Syntax `raise` Exception Type (error_message) Exception Object
Message (String)

Example: raise a name error

```
raise NameError('My Error Messgae')
```

⊗ 0.2s Python

```
-----  
-----  
NameError                                Traceback  
  ck (most recent call last)  
  Cell In[33], line 1  
----> 1 raise NameError('My Error Messgae')  
  
NameError: My Error Messgae
```

Raising Exceptions

Example: generate a fibonacci sequence

```
def fibonacci(N):  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L  
print(fibonacci(10))  
print(fibonacci(-10))
```

✓ 0.2s

[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
[]

- Input value cannot be negative
- Not raising any error
- The empty list may cause problem later

Raising Exceptions

Raise a ValueError to inform the programmer that a negative N is not supported

```
def fibonacci(N):  
    if N < 0:  
        raise ValueError("N must be non-negative")  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L  
print(fibonacci(10))  
print(fibonacci(-10))
```

⊗ 0.2s

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[7], line 11  
      9     return L  
     10 print(fibonacci(10))  
----> 11 print(fibonacci(-10))  
  
Cell In[7], line 3, in fibonacci(N)  
      1 def fibonacci(N):  
      2     if N < 0:  
----> 3         raise ValueError("N must be non-negative")  
      4     L = []  
      5     a, b = 0, 1  
  
ValueError: N must be non-negative
```

Raising Exceptions

A simple approach: raise an AssertionError

Syntax Expression returning a Boolean

```
assert condition, "error message"
```

Test if the given condition is satisfied

If True, do nothing

If False, raise AssertionError with the given error message

Raising Exceptions

Easy way: raise an AssertionError

```
a = 2
assert a == 1, 'a should be 1'
print(a)
```

⊗ 0.0s

Python

```
-----
AssertionError                                Traceback (most recent call last)
/var/folders/1p/9bcd1_s73b6cmjvy82yc_vr0000gn/T/ipykernel_99575/72001909.py i
n <module>
   1 a = 2
----> 2 assert a == 1, 'a should be 1'
   3 print(a)

AssertionError: a should be 1
```

If False,

- Raise AssertionError with the given error message

```
a = 1
assert a == 1, 'a should be 1'
print(a)
```

✓ 0.0s

Python

If True

- Do nothing

1

Update the code with an **assert** statement

```
def fibonacci(N):  
    if N < 0:  
        raise ValueError("N must be non-negative")  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L  
print(fibonacci(10))  
print(fibonacci(-10))
```

⊗ 0.2s



Function

Where we've been:

- Data types (integers, strings, lists, dicts, ...).
- Control flow (if/else, for/while loops, ...).
- Writing simple, linear scripts to perform a task.

Where we're going:

- Writing complex, organized, and reusable code.
- Building systems that can be maintained and scaled.
- This is the bridge from "writing a script" to "engineering an AI application".

Functions are the pillars of this bridge.

What is a Function? The Core Building Block

A function is a **reusable, named block of code** designed to perform a specific task.

Analogy: A Coffee Machine

- **Inputs (Arguments):** You provide coffee beans and water.
- **Function (The Machine):** It performs a process (grinds, heats, brews).
- **Output (Return Value):** You get a cup of coffee

inputs $\Rightarrow$ function $\Rightarrow$ outputs

The Purpose of Functions

Functions structure code around key principles to improve clarity, efficiency, and maintainability.

- **Modularity (DRY Principle):** Decompose complex problems into smaller, manageable, and self-contained units.
- **Reusability:** Define logic once and apply it in multiple contexts, which saves time and reduces redundancy.
- **Readability:** Make code self-documenting. For example, `compute_loss(predictions, actuals)` is far more intuitive than the raw mathematical operations.
- **Abstraction:** Hide complex implementation details behind a simple interface. The user only needs to know **what** a function does, not **how** it works.

AI Example: A neural network layer is a function. It encapsulates complex matrix operations, accepting inputs and producing outputs without exposing its internal logic

Anatomy of a Function

Let's dissect the syntax for defining a function in Python.

```
1  # --- 1. Keyword  2. Name  3. Parameters  -----
2  def greet(first_name, last_name):
3      """
4      4. Docstring: Explains what the function does.
5      """
6  # --- 5. Body: Indented code that performs the task ---
7      full_name = first_name + " " + last_name
8      message = "Hello, " + full_name + "!"
9  # -----
10     return message # 6. Return: Sends a value back
```

- **def:** The keyword that starts a function definition.
- **Name:** A unique identifier for the function (e.g., greet).
- **Parameters:** Placeholders for the data the function need to receive.
- **Docstring:** An optional string that describes the function's purpose.
- **Body:** The block of indented code that executes when called.
- **return:** The keyword that exits the function and optionally sends back a value

Defining and Calling Functions

Define with `def`; call by name with parentheses.

```
1  # --- Definition ---
2  def calculate_accuracy(predictions, labels):
3      """Calculates classification accuracy."""
4      correct = 0
5      for pred, label in zip(predictions, labels):
6          if pred == label:
7              correct += 1
8      return correct / len(labels)
9
10 # --- Calling ---
11 model_preds = ['cat', 'dog', 'cat', 'cat']
12 true_labels = ['cat', 'cat', 'cat', 'dog']
13 acc = calculate_accuracy(model_preds, true_labels)
14 print(f"Model accuracy: {acc:.2f}") # Model accuracy:
    0.50
```

Variable Scope: The LEGB Rule

Scope determines where a variable can be accessed.

Python searches for variables in this order: **Local** → **Enclosing** → **Global** → **Built-in (LEGB)**

- **Local:** Variables defined inside the current function.
- **Enclosing:** Variables in the local scope of enclosing functions (for nested functions).
- **Global:** Variables defined at the top level of a script/module.
- **Built-in:** Names pre-assigned in Python (e.g., print, len)

```
1 x = "global" # Global scope
2 def outer_func():
3     y = "enclosing" # Enclosing scope
4     def inner_func():
5         z = "local" # Local scope
6         print(z, y, x) # Accesses all three scopes
7     inner_func()
8 outer_func() # Prints: local enclosing global
```

Scope: Name Shadowing

Name shadowing: If a local variable has the same name as a variable in an outer scope, the innermost variable takes precedence

```
1 value = 10 # Global variable
2 def foo():
3     value = 3 # Local variable, shadows the global one
4     print(f"Inside foo: {value}")
5     return value
6 print(f"Before calling foo: {value}")
7 foo_result = foo()
8 print(f"After calling foo: {value}") # Global value is
   unchanged
9 print(f"Foo returned: {foo_result}")
```

Output:

```
Before calling foo: 10
Inside foo: 3
After calling foo: 10
Foo returned: 3
```

■ ■ ■ Modifying Outer Scopes: global and nonlocal

To modify a variable in an outer scope, you must explicitly declare it **global** keyword:

- Modifies a variable in the global scope.
- Use sparingly! It can make code hard to debug.

nonlocal keyword:

- Modifies a variable in the nearest enclosing scope (but not global)

```
1 TRAINING_EPOCH = 0
2 def run_training_step():
3     global TRAINING_EPOCH
4     # ... run one step ...
5     TRAINING_EPOCH += 1
```

```
1 def create_counter():
2     count = 0
3     def increment():
4         nonlocal count
5         count += 1
6         return count
7     return increment
```

Return Statement

The return statement **immediately exits** the function and **sends a value back** to the caller

Early Exit Pattern (Guard Clause)

```
1 def find_first_high_confidence(predictions, threshold=0.9):
2     for pred in predictions:
3         if pred['confidence'] >= threshold:
4             return pred # Exit as soon as we find one!
5     return None # Return None if loop finishes (no match)
```

Returning Multiple Values

- You can return multiple values separated by commas. Python automatically packs them into a **tuple**.

```
1 def get_model_stats(history):
2     loss = history['loss'][-1]
3     accuracy = history['accuracy'][-1]
4     return loss, accuracy # Returns a tuple: (loss, accuracy)
5 # Unpack the returned tuple into variables
6 final_loss, final_acc = get_model_stats(model_history)
```


Docstrings

Document string: A very first string after function header

- Can be any string (created with ' ', " ", ''' ''', """ """)
- Access it by the `__doc__` attribute of function (functions are objects, with attributes)
- Used by `help()` function and other tools to generate documentation
- A good docstring states **purpose, parameters, returns, and exceptions**.

```

1 def preprocess_image(image, target_size=(224, 224)):
2     """Resizes and normalizes an image for model input.
3     Args:
4         image: A NumPy array representing the image.
5         target_size (tuple): The desired (width, height)
6         .
7     Returns:
8         The preprocessed image as a NumPy array.
9     """
10    # ... implementation ...
11    return processed_image

```

Type Hint

- **Optional annotations** for arguments and return values.
- Not enforced at runtime; they are for developers and tools (IDEs, static analysis).
- Crucial for large, collaborative AI projects to prevent bugs.

```
1  from typing import List, Tuple, Dict, Any
2  def predict(model: Any,
3              data: List[Dict[str, float]]) -> List[str]:
4      """
5      Generates predictions for a batch of data.
6      Args:
7          model: A trained machine learning model object.
8          data: A list of dictionaries, where each dict is a
9                feature vector.
10     Returns:
11         A list of predicted labels (strings).
12     """
13     # ... implementation ...
14     return predictions
```

Task: Write a function **normalize_features**.

- It should accept a list of numbers (e.g., pixel values, sensor readings).
- It should find the minimum and maximum values in the list.
- It should return a new list where each value is scaled to be between 0 and 1 using the formula:

$$\text{normalized_value} = \frac{\text{value} - \text{min_val}}{\text{max_val} - \text{min_val}}$$

- Include a docstring and type hints.
- What should happen if `max_val == min_val`? Handle this edge case

Example call:

```
1 data = [10, 20, 30, 40, 50]
2 normalized_data = normalize_features(data)
3 # expected: [0.0, 0.25, 0.5, 0.75, 1.0]
```

Parameters vs. Arguments

A subtle but important distinction:

- **Parameters** are the named variables in a function's **definition**. They act as placeholders for the inputs.
- **Arguments** are the actual **values** you pass to the function when you **call** it

```
1 # 'a' and 'b' are PARAMETERS
2 def add(a, b):
3     return a + b
4
5 # 5 and 10 are ARGUMENTS
6 result = add(5, 10)
```

Argument Passing: Positional vs. Keyword

Positional Arguments

- Matched to parameters in order.
- The meaning depends entirely on position

Keyword Arguments

- You explicitly name which parameter gets which value.
- The order no longer matters.
- Makes code more readable and less error-prone.

```
1 def create_user(name, age,
2                 role):
3     # ...
4     # 'john' -> name, 30 -> age,
5     # etc.
6     create_user('john', 30, '
7                 admin')
```

```
1 create_user(
2     role='admin',
3     name='john',
4     age=30
5 )
```

Argument Passing: Positional vs. Keyword

Positional Arguments

- Matched to parameters in order.
- The meaning depends entirely on position

```
1 def create_user(name, age,
2                 role):
3     # ...
4     # 'john' -> name, 30 -> age,
5     # etc.
6     create_user('john', 30, '
7                 admin')
```

Keyword Arguments

- You explicitly name which parameter gets which value.
- The order no longer matters.
- Makes code more readable and less error-prone.

```
1 create_user(
2     role='admin',
3     name='john',
4     age=30
5 )
```

Rules: Positional arguments must always come before keyword arguments in a function call

`create_user(name='john', 30, 'admin')` → **SyntaxError**.

Default Arguments

You can provide a default value for a parameter, making it optional for the caller

```
1 # AI Use Case: A flexible model training function
2 def train_model(data, epochs=10, learning_rate=0.001):
3     print(f"Training for {epochs} epochs with lr={
4         learning_rate}")
5     # ... training logic ...
6 # --- Calling the function ---
7 # 1. Use all defaults (most common case)
8 train_model(my_training_data)
9 # 2. Override 'epochs' by its position
10 train_model(my_training_data, 50)
11 # 3. Override 'learning_rate' by its keyword (more
12     readable)
13 train_model(my_training_data, learning_rate=0.01)
```

Rules: In the function definition, parameters with defaults must come after parameters without defaults

■ The Mutable Default Argument Pitfall

Warning: Never use a mutable type (like a list or dictionary) as a default argument. This is a classic Python trap!

The default object is created only once (when the function is defined) and is reused for all subsequent calls that don't provide that argument.

```
1  # WRONG WAY - The list is shared across calls
2  def add_log(message, logs=[]):
3      logs.append(message)
4      print(logs)
5  add_log("Starting process...") # >> ['Starting process...']
6  add_log("Step 1 complete.")    # >> ['Starting process...', 'Step 1
   complete.']
7  \pause
8  # CORRECT WAY - Create a new list each time if needed
9  def add_log_correct(message, logs=None):
10     if logs is None:
11         logs = []
12     logs.append(message)
13     print(logs)
14 add_log_correct("Starting process...") # >> ['Starting process...']
15 add_log_correct("Step 1 complete.")    # >> ['Step 1 complete.']
```

Variable-Length Positional Arguments: *args

The `*args` syntax allows a function to accept any number of extra positional arguments, which are collected into a **tuple**

```
1 def process_data(required1, required2, *args):
2     print(f"Required 1: {required1}")
3     print(f"Required 2: {required2}")
4     print(f"Other args: {args}") # 'args' is a tuple
5 process_data('model_A', 'dataset_X')
6 # >> Other args: ()
7 process_data('model_B', 'dataset_Y', 'option1', 123,
8             True)
9 # >> Other args: ('option1', 123, True)
10 # AI Use Case: A function to average multiple loss
11 # values
12 def average_loss(*losses):
13     if not losses: return 0
14     return sum(losses) / len(losses)
```

Variable-Length Keyword Arguments: **kwargs

The `**kwargs` syntax allows a function to accept any number of extra keyword arguments, which are collected into a **dictionary**

```
1 def create_model_config(model_type, **kwargs):
2     config = {'type': model_type}
3     # kwargs is a dictionary of all other keyword arguments
4     config.update(kwargs)
5     return config
6 # 'layers' and 'dropout' are collected into kwargs
7 config1 = create_model_config('CNN', layers=4, dropout=0.5)
8 # >> {'type': 'CNN', 'layers': 4, 'dropout': 0.5}
9 # 'optimizer', 'lr', and 'beta' go into kwargs
10 config2 = create_model_config('Transformer', optimizer='Adam', lr
    =0.0001, beta=0.9)
11 # >> {'type': 'Transformer', 'optimizer': 'Adam', ...}
```

AI Use Case: Perfect for passing flexible configuration parameters to model layers, optimizers (Adam(lr=..., beta1=...)), or plotting functions.

Unpacking Arguments with * and **

You can also use * and ** when calling a function to unpack iterables(lists, tuples) and dictionaries into arguments. This is the inverse of collecting with *args/**kwargs.

```
1 def plot_metrics(epoch, accuracy, loss, val_accuracy, val_loss):
2     print(f"E:{epoch} | Acc:{accuracy:.2f} | Loss:{loss:.2f} | "
3           f"Val Acc:{val_accuracy:.2f} | Val Loss:{val_loss:.2f}")
4 # Data is in a list/tuple
5 epoch_data = [5, 0.92, 0.15, 0.88, 0.21]
6 plot_metrics(*epoch_data) # Unpacks list into positional args
7 # Data is in a dictionary
8 metrics_dict = {
9     'accuracy': 0.95, 'loss': 0.11,
10    'val_accuracy': 0.91, 'val_loss': 0.18
11 }
12 # Unpacks dict into keyword args
13 plot_metrics(10, **metrics_dict)
```

Keyword-Only and Positional-Only

Python allows you to enforce how arguments must be passed, creating more robust APIs.

- Keyword-Only (after *): Forces arguments to be passed by keyword. Improves clarity, especially for flags
- Positional-Only (before /): Prevents arguments from being passed by keyword. Useful when parameter names might change.

```
1 # 'verbose' and 'log_file'  
    must use keywords  
2 def process_files(*files,  
    verbose=False, log_file  
    =None):  
3     # ...  
4 process_files('a.txt',  
    verbose=True) # OK  
5 # process_files('a.txt',  
    True) # TypeError!
```

```
1 # 'x' and 'y' must be  
    positional  
2 def my_func(x, y, /):  
3     # ...  
4 my_func(1, 2) # OK  
5 # my_func(x=1, y=2) #  
    TypeError!
```

The Complete Parameter Order

When defining a function, parameters must be in this specific order

Function Definition Syntax

```
def name(pos_only, /, standard, *args, kw_only, **kwargs):
```

- Positional-Only: Can only be passed by position. (Ends with /)
- Standard: Can be passed by position or keyword. (e.g., data, epochs=10)
- *args: Collects all extra positional arguments.
- Keyword-Only: Must be passed by keyword. (Come after *args or a bare *)
- **kwargs: Collects all extra keyword arguments.

You don't need to use all of them, but those you use must be in this order

Why This Order Matters

Intuition: The order prevents ambiguity in how arguments are matched.

- Positional arguments first, so Python can match left-to-right.
- Then defaults and `*args`, so extra values have a clear place.
- Keyword-only arguments must be named, avoiding confusion.
- Finally `**kwargs`, which catches everything else.

What could go wrong without order?

```
1 # Ambiguous: is 'admin' a keyword-only arg or part of *args?
2 def create_user(*args, role, **kwargs): ...
3 create_user('john', 'admin') # Clear with enforced order
4
5 # Another ambiguity: where to assign values if order is not fixed?
6 def f(a=1, *args, b, **kwargs): ...
7 f(2, 3, b=4) # Without order, is 3 for *args or b?
```

Takeaway: The fixed order makes function definitions **predictable, readable, and safe**.

Goal: Build a flexible AI training configuration function using Python's argument-passing features.

Tasks

- 1. Initial Setup:** Define `configure_training(model_name, dataset, epochs=10)`. The function should print its arguments. Call it with just the two required ones.
- 2. Optimizer Settings:** Add `**kwargs` to your function to capture extra optimizer settings (e.g., `learning_rate=0.01`). Print the settings dictionary. Call it with a custom learning rate.
- 3. Final Polish & Unpacking:** Refactor to make `epochs` a keyword-only argument. Create a dictionary of optimizer settings and call the function, unpacking the dictionary into arguments using `**`



Anonymous(Lambda) Functions

Functions are First-Class Objects

- Function is an object
- Function name can be regarded as a variable storing the object

```
def add(a, b):  
    str(1)  
    c = a + b  
    return c  
print(add)
```

✓ 0.0s

```
<function add at 0x10b0b5280>
```

Functions are First-Class Objects

- In Python, functions are treated like any other variable (e.g., an integer or a list).
- This is a powerful concept called "first-class functions".
- This means you can:
 - **Assign a function to a variable.**
 - **Pass a function as an argument** to another function (higher-order functions).
 - **Return a function** from another function.
 - **Store functions in data structures** like lists or dictionaries.

```
1 def square(x): return x * x
2 def cube(x): return x * x * x
3 # 1. Assign to a variable & store in a dict
4 math_ops = {'sq': square, 'cb': cube}
5 print(math_ops['sq'](5)) # Output: 25
6 # 2. Pass as an argument
7 def apply_operation(op, value): return op(value)
8 print(apply_operation(cube, 10)) # Output: 1000
```

Advanced Concept: Anonymous (Lambda) Function

The Problem: Sometimes you need a small, simple function for a single use. Writing a full def statement feels verbose.

The solution: A lambda function

Syntax: 01. `lambda parameters: return_expression`

- A lambda is defined on a single line.
- It can have multiple arguments, but only one expression.
- The result of the expression is implicitly returned.

Variable (name) Parameter

```
square = lambda x: x * x
```

Returned value

```
print(square(4))
```



```
def square(x):  
    return x * x  
print(square(4))
```

Call the function with the variable

Key takeaway: Use lambdas for short, simple operations, often when passing a function as an argument. For complex logic, always use a def

■ ■ ■ Lambdas in Action: map and filter

Lambdas are most powerful when used with higher-order functions.

map(function, iterable): Applies a function to every item of an iterable.

```
1 # AI Example: Convert raw scores (logits) to
  probabilities
2 logits = [-2.2, -1.4, 0.8, 2.0]
3 probabilities = list(map(lambda x: 1 / (1 + 2.718** -x),
  logits))
```

filter(function, iterable): Returns items from an iterable for which the function returns True

```
1 # AI Example: Filter out low-confidence predictions
2 predictions = [{'conf': 0.98}, {'conf': 0.65}, {'conf':
  0.91}]
3 high_conf = list(filter(lambda p: p['conf'] > 0.9,
  predictions))
```

■ Lambdas in Action: sorted

Lambdas are most powerful when used with higher-order functions.

`sorted(iterable, key=..., reverse=False):`

- Use a lambda key to sort complex objects.
- Set `reverse=True` for descending order

```
1 # Example: sort detections by confidence (descending)
2 detections = [
3     {'label': 'person', 'confidence': 0.85, 'box': [10,10,50,50]},
4     {'label': 'car',     'confidence': 0.98, 'box': [60,20,100,80]},
5     {'label': 'person', 'confidence': 0.91, 'box': [15,12,55,52]},
6 ]
7 sorted_by_conf = sorted(detections, key=lambda d: d['confidence'],
                        reverse=True)
```

- `d` will iterate through elements in `detections`

Lambdas in Action: reduce

Lambdas are most powerful when used with higher-order functions.

`reduce(function, iterable[, initializer])` (from `functools`)

- Combine a sequence into a single value (e.g., sum, product, min).
- Prefer built-ins like `sum` when available; use `reduce` for custom folds

```
1 from functools import reduce
2 # Example: product via reduce
3 nums = [1, 2, 3, 4]
4 prod = reduce(lambda a, b: a * b, nums, 1)
```

- The last line performs for $t \in \{0,1,2,3\}$: $a_{t+1} = a_t \times b_t$, where $a_0 = 1$, $b_t = \text{nums}[t]$.

Lambdas v.s. List Comprehensions

For many map and filter operations, a list comprehension is often considered more "Pythonic" and readable

map equivalent:

```
1 # Using map and lambda
2 numbers = [1, 2, 3, 4]
3 squared_map = list(map(lambda x: x**2, numbers))
4 # Using a list comprehension (often preferred)
5 squared_comp = [x**2 for x in numbers]
```

filter equivalent:

```
1 # Using filter and lambda
2 numbers = [1, 2, 3, 4, 5, 6]
3 evens_filter = list(filter(lambda x: x % 2 == 0, numbers))
4 # Using a list comprehension
5 evens_comp = [x for x in numbers if x % 2 == 0]
```

Guidelines: If you are creating a list, comprehensions are usually clearer. Use map and filter when you need to pass the function itself, not just the results.

Goal: Use lambda in realistic AI mini-tasks; compare with comprehensions/def alternatives

Tasks

- Sigmoid probabilities: Given logits, use **map + lambda** to compute $\sigma(x) = 1/(1 + e^{-x})$ for each.
- Filter predictions: From a list of dicts with {"label": str, "conf": float}, keep only items with `conf >= 0.9` using **filter + lambda**.
- Rank object detection results: Use **sorted** with a **lambda key** to sort detections by confidence (descending). Break ties by larger box area.
- Rewrite one subtask using a list comprehension or a normal def function. Which version is clearer and why?

Advanced Concept: Closures

A **closure** occurs when a nested function remembers and has access to variables from its enclosing scope, even after the outer function has finished executing.

```
1 def make_multiplier(n):
2     # 'n' is a "free variable" from the enclosing scope
3     def multiplier(x):
4         return x * n # multiplier remembers 'n'
5     return multiplier
6 # 'times3' is a closure that remembers n=3
7 times3 = make_multiplier(3)
8 # 'times5' is a closure that remembers n=5
9 times5 = make_multiplier(5)
10 print(times3(10)) # Output: 30
11 print(times5(10)) # Output: 50
```

AI Connection: Closures are the foundation for concepts like decorators and are used in frameworks to create specialized, configured functions (e.g., creating loss functions with specific weights).

Advanced Concept: Decorators

A **decorator** is a function that takes another function as input, adds some functionality to it, and returns the modified function, without permanently changing the original function's code.

```
1 # A simple decorator to time a function's execution
2 import time
3 def timer(func):
4     def wrapper(*args, **kwargs):
5         start_time = time.time()
6         result = func(*args, **kwargs)
7         end_time = time.time()
8         print(f"{func.__name__} took {end_time - start_time:.4f}s")
9         return result
10    return wrapper
11 @timer
12 def train_model():
13     """Simulates a time-consuming training process."""
14     time.sleep(2)
15     print("Model training complete.")
16 train_model() # This will now be timed automatically!
```

The output: Model training complete.
 train_model took 2.0011s

Goal: Implement a parameterized closure and a reusable decorator, then compose them.

Task:

- Closure: Write `make_power(n)` that returns a function raising its input `x` to the power `n`.
- Decorator: Write `log_call(func)` that prints the function name, arguments, and result each time it is called.
- Compose: Use the decorator to log calls to a closure produced by `make_power`.

Questions:

- Why does `power` still see `n` after `make_power` returns?
- How does the decorator add logging without changing the closure logic?

Exceptions

- Exception Hierarchy
- Exception Handling: try, except, else, finally, raise, assert

Functions

- Fundamentals: define/call, variable scope, return statement, docstring, type hint
- Parameters/Arguments: positional v.s. keyword, default, variable length (*args, **kwargs), Keyword-only and Positional-Only(* and /), parameter order
- Lambda Functions: use with higher-order functions (map, filter, sorted, reduce)

Python Programming with Artificial Intelligence

Thank you for listening!